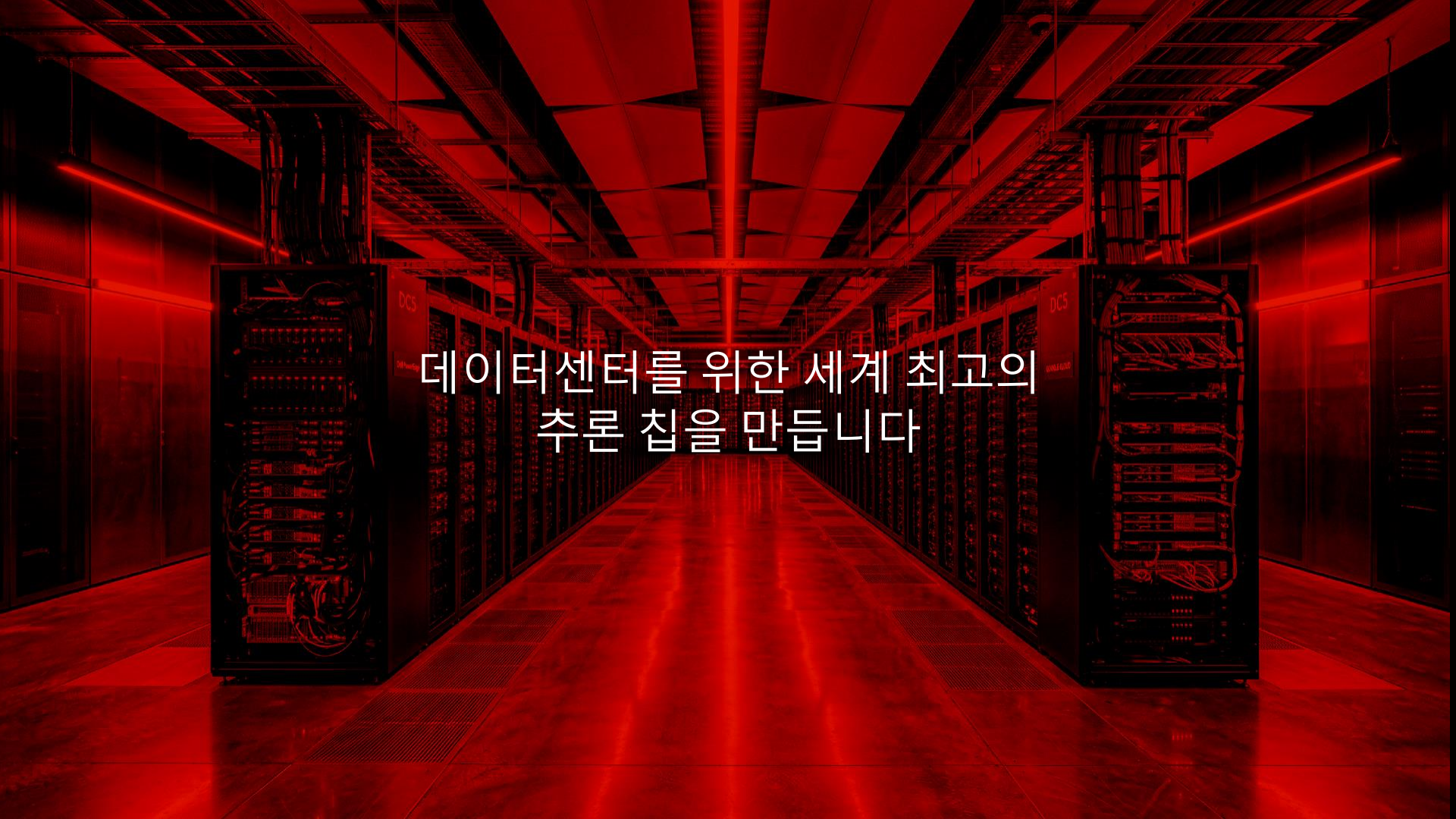


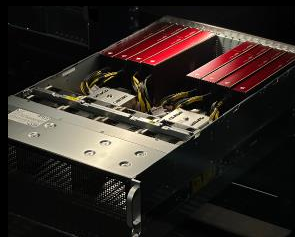
FURIOSA 

RENEGADE





데이터센터를 위한 세계 최고의
추론 칩을 만듭니다



2017년

2022년 6월

2024년 5월

2026년 1월

FuriosaAI 창립

Gen 1 Vision NPU
양산

Gen 2 RNGD 실리콘 입고

RNGD 양산 개시

AlexNet
2012년 9월

알파고 vs. 이세돌
2016년 3월

Transformer
2017년 6월

ChatGPT 모멘트
2022년 11월

DeepSeek R1 / 프론티어
오픈 모델의 부상
2025년 1월

에이전틱 AI 시대 개막
2026년-

에이전틱 AI 시대, 폭발적으로 증가하는 토큰 수요





100+ GW

2030년까지 신규 구축되는 데이터센터

70%

추론 전용 데이터센터 비중

McKinsey & Company 2030년 전망: 추론 93.3 GW vs 학습 62.2 GW
OpenAI 공개 수치: 추론 3 GW / 학습 2 GW

추론 시대를 위해 설계된 Furiosa RNGD

512 TFLOPS

64 TFLOPS (FP8) × 8
Processing Element

48 GB

메모리 용량

256 MB SRAM

384 TB/s
온칩 대역폭

1.5 TB/s

메모리 대역폭

180 W TDP

공랭식 데이터센터 대응

2 × HBM3

CoWoS-S



tsmc

SK hynix

GUC
The NVIDIA GUC

ASUS

Furiosa NXT RNGD 서버

8× RNGD

장착 카드 수

4 petaFLOPS

512 TFLOPS × 8장

384 GB

HBM3 용량

12 TB/s

메모리 대역폭

3 kW

소비 전력



HPE

arm



DELL

GIGABYTE

Lenovo

Multi-Batch LLM Generation

0

Max TPS

0

TPS

128

Batches

0

Avg. TPS



EXAONE-4.0-32B-FP8

8x

실제 운영 환경에서 30~100% 더 높은
전력 효율, 고객 검증 완료

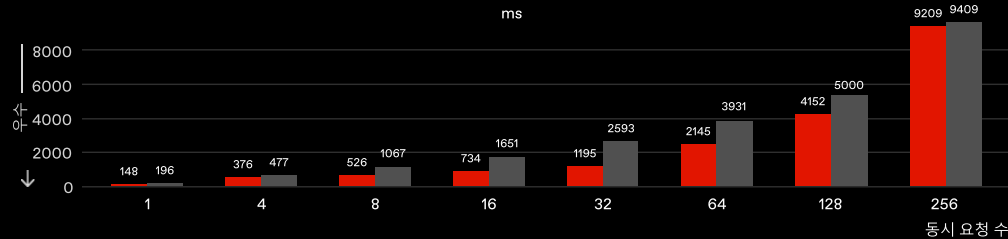
Qwen 3-32B, FP8, 1K / 1K

■ RNGD x4 (3월 13일)

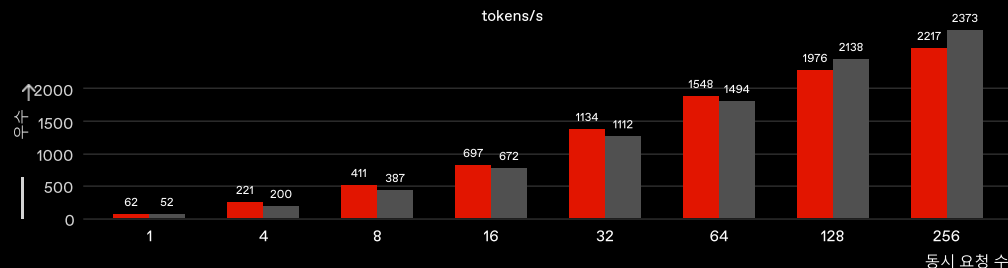
■ RTX PRO 6000 x4 (vLLM 0.18.0)

참고: RTX PRO 6000은 RunPod 환경에서 측정

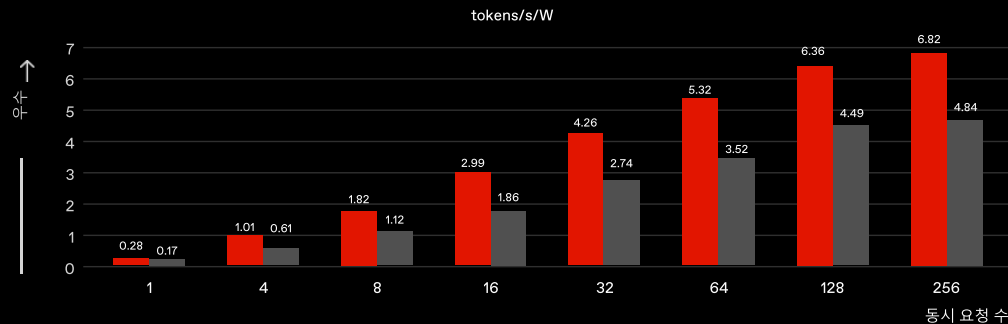
첫 토큰 생성 시간 (TTFT)



출력 처리량

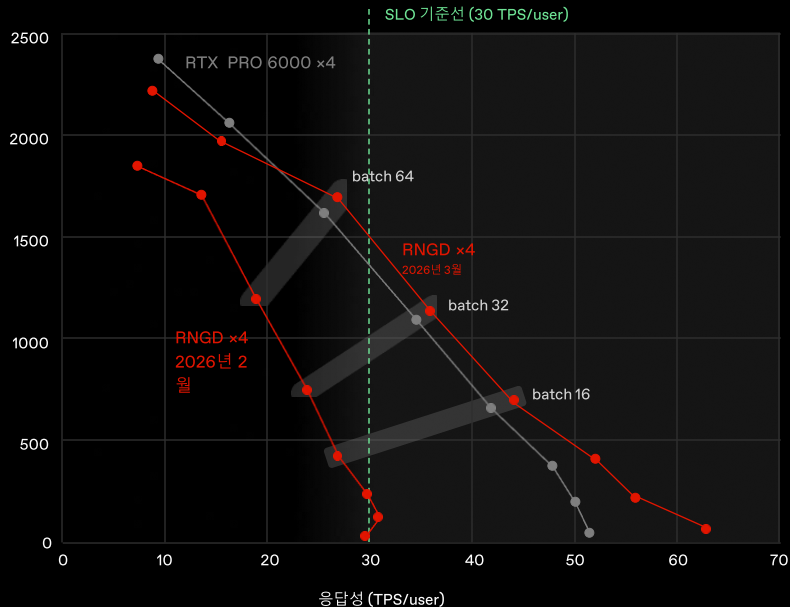


전력 효율

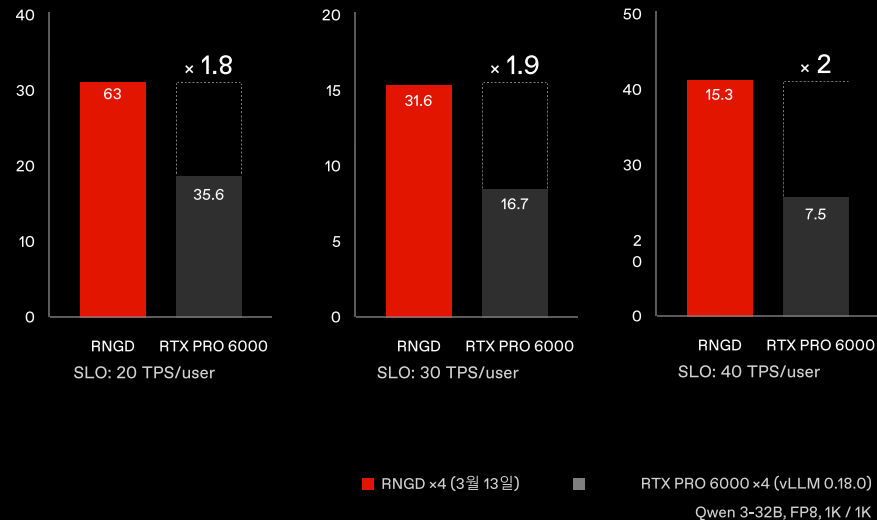


목표 SLO 기준, 랙당 최대 2배 더 많은 사용자 수용

출력 처리량 (tokens/s)



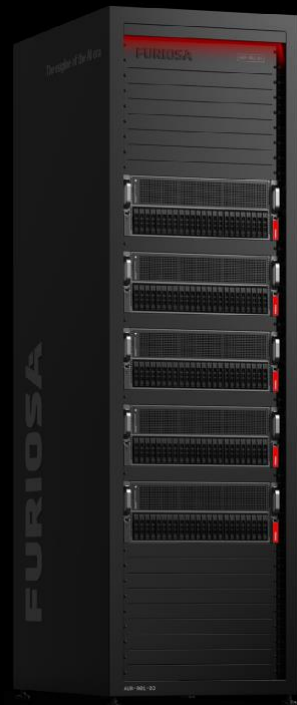
Users/kW



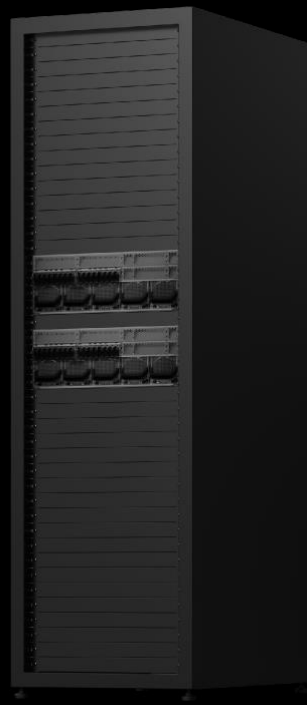
랙 단위 경제성: RNGD는 4배 더 많은
추론 용량을 제공합니다

랙당 최대 서버 수	5x	2x
서버 소비 전력 (kW)	3 kW	7.5 kW
랙당 처리량	26,400 tokens/s	6,600 tokens/s
랙당 최대 사용자 수	880	220

 NXT RNGD



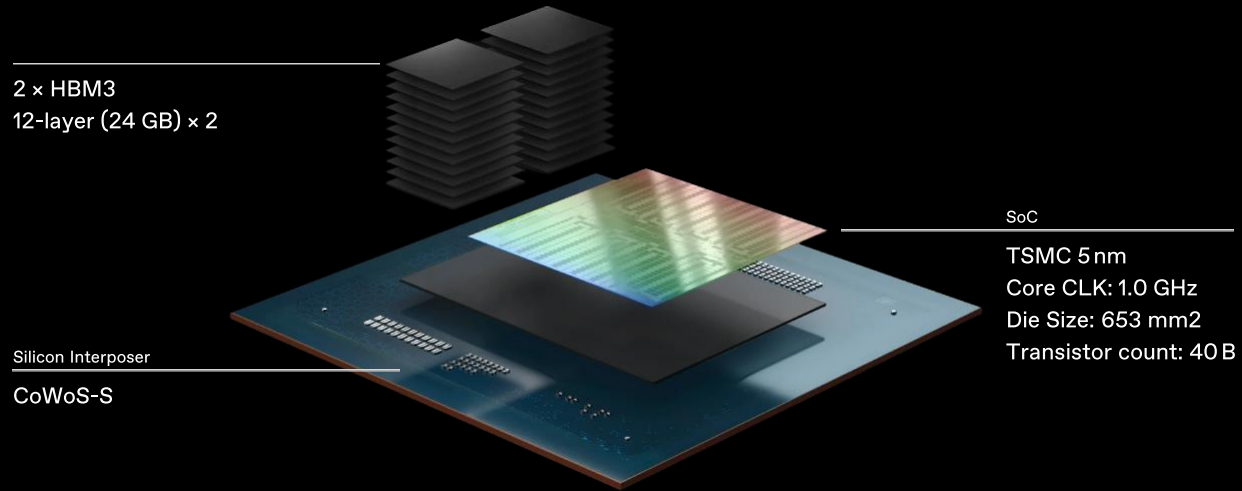
RTX PRO 6000



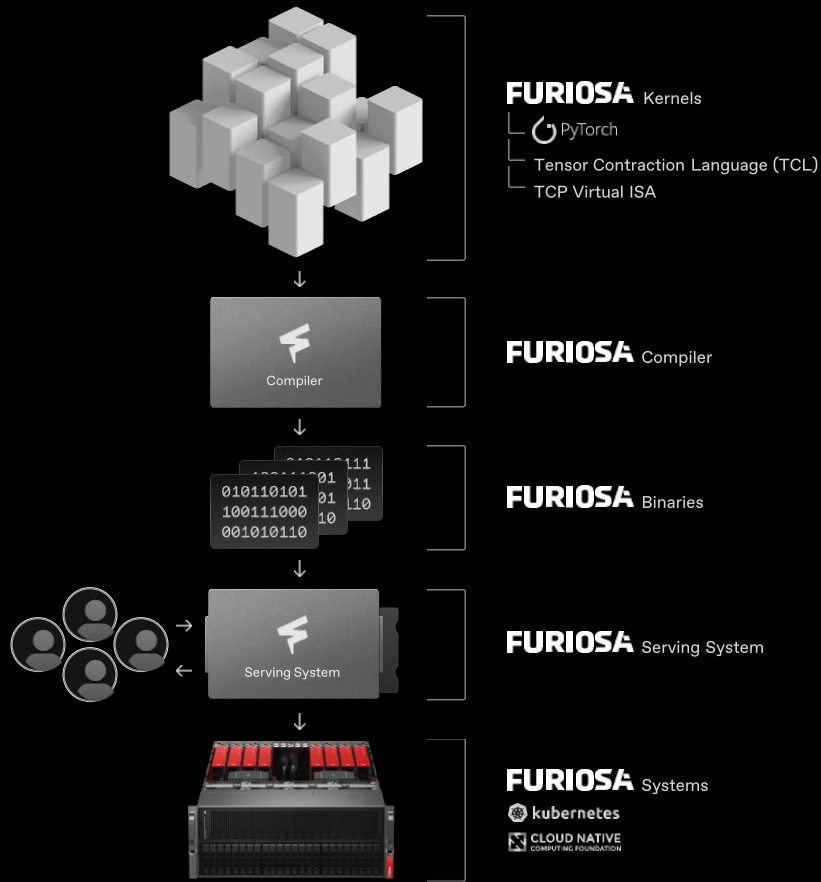
Tensor Contraction Processor: 추론 워크로드에 최적화된 지점



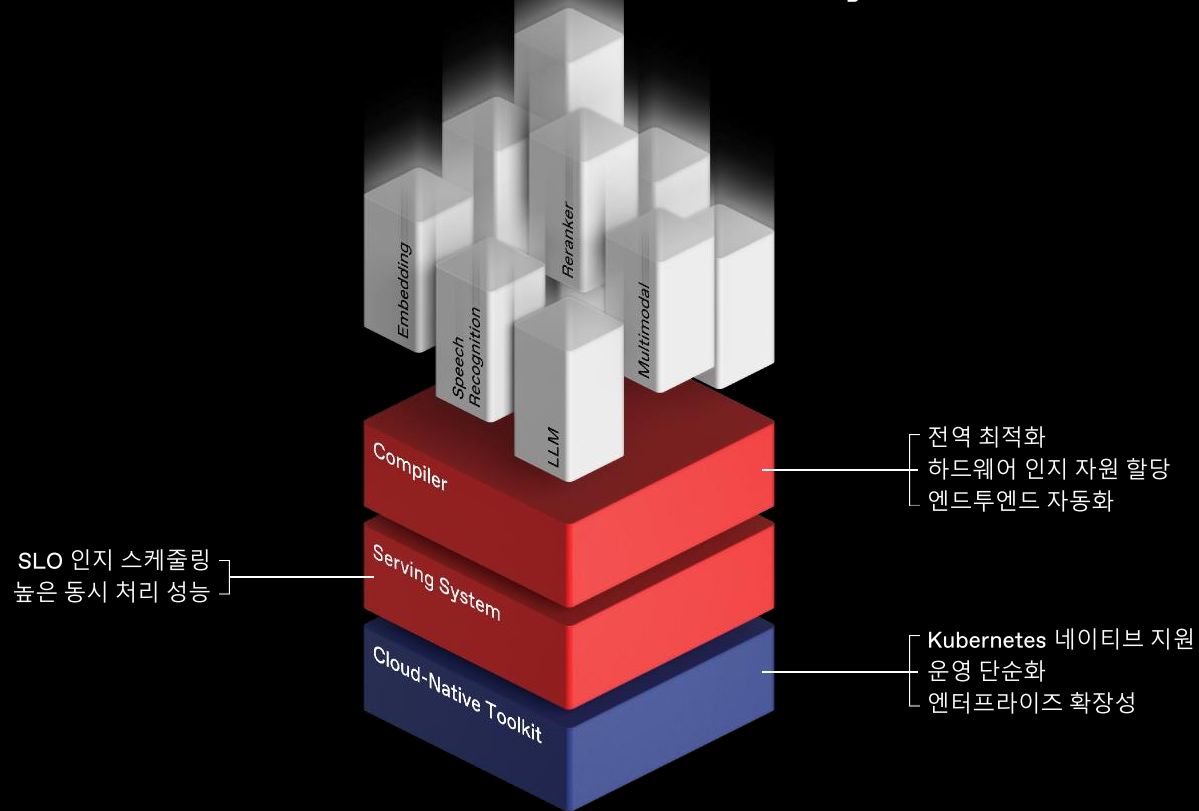
첨단 아키텍처와 하드웨어 기술을 하나의 칩에 집약



풀스택 소프트웨어 플랫폼으로 완성
하는 RNGD의 가치



성능·안정성·확장성을 갖춘 Production Ready 소프트웨어



지원 모델 예시

OpenAI gpt-oss 120b

 PyTorch 90%

```
def _convert_moe_packed_tensors(
    blocks,
    scales,
    *,
    dtype: torch.dtype = torch.bfloat16,
    rows_per_chunk: int = 32768 * 1024,
) -> torch.Tensor:
    ...
    FP4_VALUES = [+0.0, +0.5, +1.0, +1.5, +2.0, +3.0, +4.0, +6.0,
                  -0.0, -0.5, -1.0, -1.5, -2.0, -3.0, -4.0, -6.0]
    lut = torch.tensor(FP4_VALUES, dtype=dtype, device=blocks.device)
    for r0 in range(0, rows_total, rows_per_chunk):
        r1 = min(r0 + rows_per_chunk, rows_total)
        blk, exp, sub = blocks[r0:r1], scales[r0:r1], out[r0:r1]
        idx_lo = (blk & 0x0F).to(torch.int)
        sub[:, 0:2] = lut[idx_lo]
        del idx_lo
        idx_hi = (blk >> 4).to(torch.int)
        sub[:, 1:2] = lut[idx_hi]
        del idx_hi
        torch.indexexp(sub, exp, out=sub)
        del blk, exp, sub
    out = out.reshape(*prefix_shape, G, B * 2).view(*prefix_shape, G * B * 2)
```

GPU-Equivalent Developer Experience

 TCL 9%

```
def mxfp4_dequantize(
    input: [B, E]/f4_e2,
    scale: [B]/f8,
) -> [B, E]/bf16:
    T_Axis = 16
    look_up_tbl: [T]/f8_e4 = vector(
        [0.0, 0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 6.0,
         -0.0, -0.5, -1.0, -1.5, -2.0, -3.0, -4.0, -6.0])
    scale_f32: [B]/i32 = tk.Elementwise(
        %0 = read(scale)
        %1 = ve.exec(if %0 == 10 { %0 - _fxp_sat f127 } else { %0 + _fxp_sat f129 })
        %2 = ve.exec(%1 << i23)
    )
    output: [B, E]/bf16 = tk.Elementwise(
        %0 = read(input, table_lookup=look_up_tbl)
        %1 = read(scale_f32)
        %2 = ve.exec(%0 & 1 0x7fffff) # &_i: interpret output as i32
        %3 = ve.exec(%2 + _fxp_sat %1)
        %5 = ve.exec(if %0 != f0 { %3 } else { %3 * _i i0 })
        %6 = ve.exec(%5 min_i 0x7f800000)
        %7 = ve.exec(%6 max_i 0x00000000)
        %9 = ve.exec(if %0 >= f0 { %7 } else { %7 + _fxp 0x80000000 })
        %10 = ve.exec(%9 as_f) # as_f: interpret %9 as f32
        write.to([B, E]/bf16)
    )
    return output
```

Tensor and HBM Programming

 TCP Virtual ISA 1%

```
pub(super) fn cache_kv(
    ctx: &mut Context,
    attn_k: &HbmTensor<bf16, Chip, m[T, K]>,
    attn_v: &HbmTensor<bf16, Chip, m[T, K]>,
    k_scatter_index: &HbmTensor<i32, Chip, m[1, T]>,
    v_scatter_index: &HbmTensor<i32, Chip, m[1, T]>,
    out_k_cache: &mut HbmTensor<bf16, Chip, m[C, 1, N, D]>,
    out_v_cache: &mut HbmTensor<bf16, Chip, m[C, 1, N, D]>,
){
    // V scatter index path.
    // Load V scatter indices from HBM to DM.
    let v_idx_dm: DmTensor<i32, Chip, Cluster, m[T / 32, 1 # 8], m[T % 32]> =
        v_scatter_index.to_dm(&mut ctx.dma, 0x0);
    // Reorder index tiles to match the downstream scatter-friendly layout.
    let v_idx_it: DmTensor<i32, Chip, Cluster, m[T / 32, T / 4 # 8], m[T % 4 #
        32]> = ctx
        .main
        .begin(v_idx_dm.view())
        .fetch<i32, m[T / 4 # 8], m[T % 4]>0
        .switch<cm[T / 32, T / 4 # 8], m[1 # 8]>(SwitchConfig::InterTranspose {
            slice: 8,
            slice0: 1,
            time0: 1,
        })
        .collect<cm[1 # 8], m[T % 4]>0
        .commit(0x100);
}
```

Compute and SRAM Programming

→ <https://developer.furiosa.ai/furiosa-opt/book>

기존 GPU를 그대로 대체 — 지금 바로 스택 전환이 가능합니다

OpenAI

Compatible API endpoint

```
$ uv pip install furiosa-llm --torch-backend=auto
$ furiosa-llm serve furiosa-ai/EXAONE-4.0-32B-FP8
```

Server

```
$ curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "EMPTY",
  "messages": [{"role": "user", "content":
    "What is the capital of France?"}]
}' \
| python -m json.tool
```

Client

vLLM

Compatible Python module

```
from vllm.furiosa_llm import LLM, SamplingParams

# Load the model from Hugging Face model hub
llm = LLM("furiosa-ai/EXAONE-4.0-32B-FP8")

# You can specify various parameters for text generation
sampling_params = SamplingParams(min_tokens=10, top_p=0.3, top_k=100)

# Prompt for the model
messages = [{"role": "user", "content": "What is the capital of France?"}]

# Generate text
response = llm.chat(messages, sampling_params)
```



kubernetes



llm-d

Cloud-native device plugin

```
apiVersion: v1
kind: Pod
metadata:
  name: npu-pod
spec:
  containers:
    - name: npu-pod
      image: ubuntu:noble
      resources:
        limits:
          cpu: "16"
          memory: "32Gi"
          furiosa.ai/npu: "4"
```

프로덕션 레디: 코드 한 줄 변경으로 오늘 바로 배포

```
from vllm import LLM
|
# Load the model from Hugging Face model hub
llm = LLM("furiosa-ai/EXAONE-4.0-32B-FP8")

# Load the model from Hugging Face model hub
messages = [{"role": "user", "content": "What is the capital of France?"}]

# Generate text
response = llm.chat(messages)
```

프론티어 오픈웨이트 모델 지원 역량

지원 모델 수 ↑



"Furiosa RNGD provides a compelling combination of benefits: excellent real-world performance, a dramatic reduction in our total cost of ownership, and a surprisingly straightforward integration."

 **LG AI Research**

Kijeong Jeon, Product Unit Lead

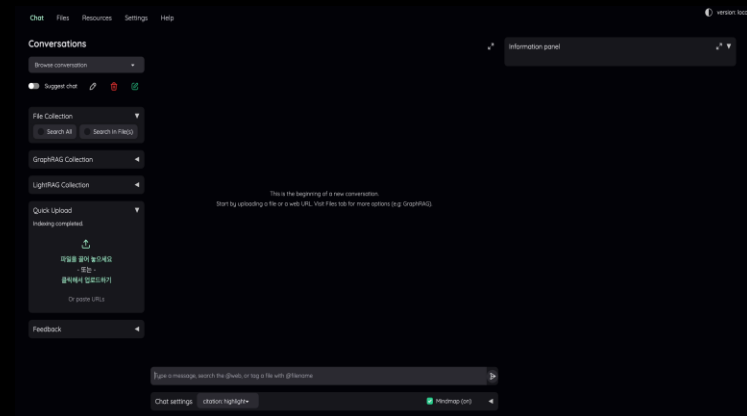
파트너 / 고객사

 **LG U+**

 **LG CNS**

GitHub에서 제공되는 레퍼런스 애플리케이션

OpenClaw

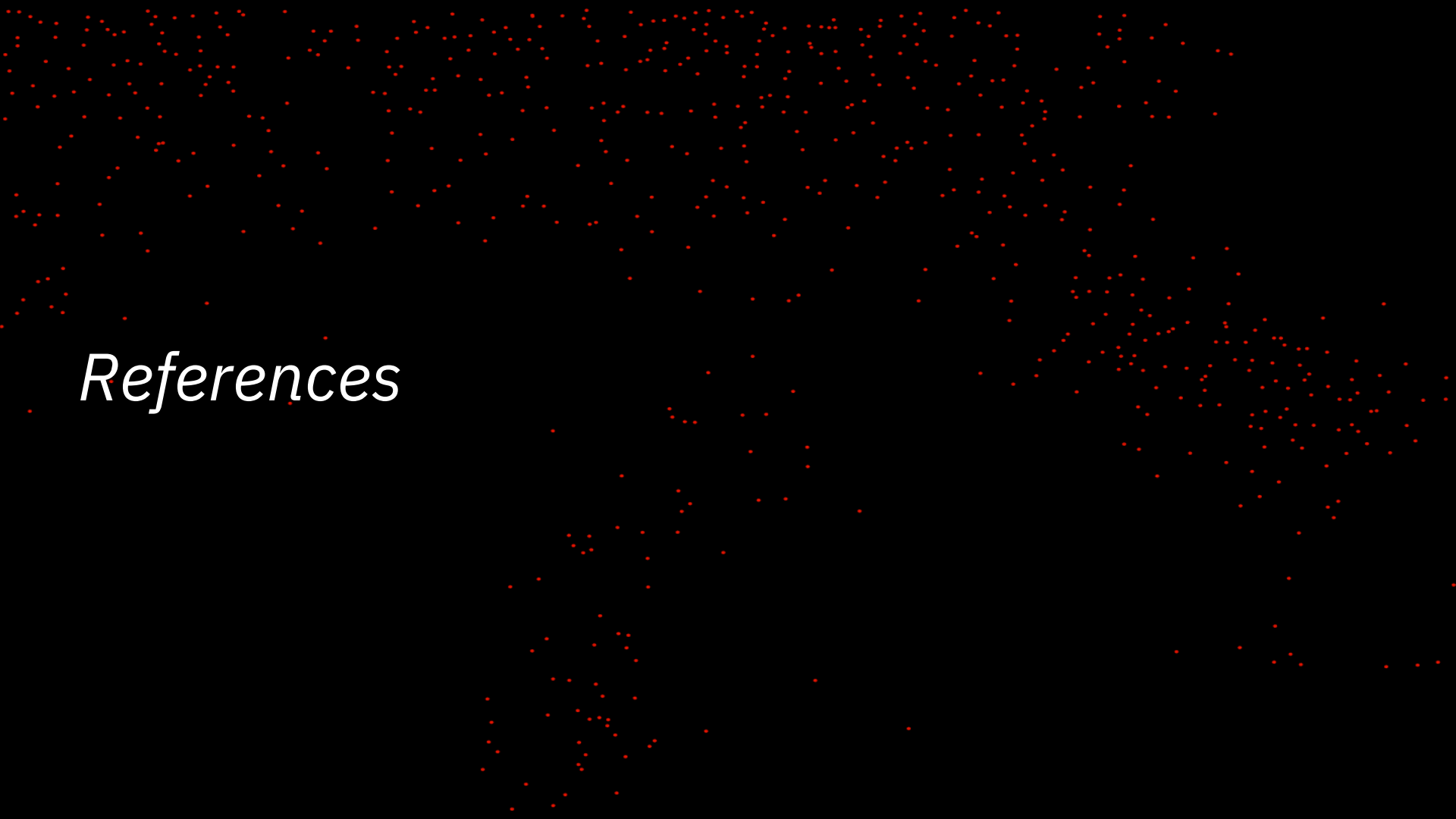
에이
전트

Kotaemon

RAG 시스템

→ github.com/furiosa-ai/furiosa-apps

FURIOSA 

The background of the slide is black, covered with a sparse, irregular distribution of small, bright orange dots. These dots are scattered across the entire frame, creating a textured, starry effect.

References

FURIOSA × SAMSUNG SDS

삼성SDS, 7월 RNGD 기반 국내 첫 NPUaaS 출시 예정

Samsung SDS
Data Center
(in Korea)



Suwon



Sangam



Dongtan

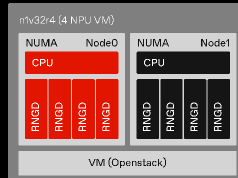
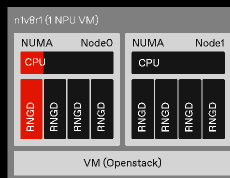


Daegu PPP
(Private cloud network)



RNGD VM (26.7 First NPUaaS among Korean CSPs)

NPUaaS



FURIOSA × LGU⁺

엔터프라이즈 고객을 위한 소버린 AI 어플라이언스 공급



엔터프라이즈 AI
풀스택

MCP

AI Orchestration

AI Agent

A2A

AI Ops



Deepnoid – 클라우드 기반 흉부 X-ray AI 판독문 생성

실증 병원

EU/MC 이대서울병원
SEOUL

SNUH 서울대학교병원
서울특별시보라매병원

인하대병원
INHA UNIV. HOSPITAL

Chest X-ray 생성형 AI 활용 연동



흉부 X-ray
이미징

PACS 연동

CXR 소견서
생성

의료영상 DICOM

powered by
FURIOSA

판독소견서 SR



Standardized

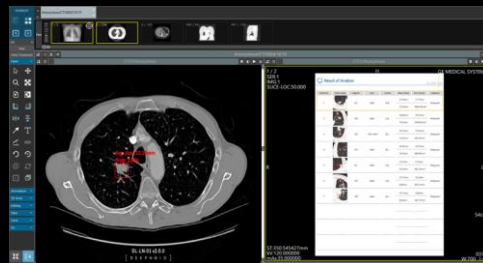
Verbose

Abbreviated

FINDINGS

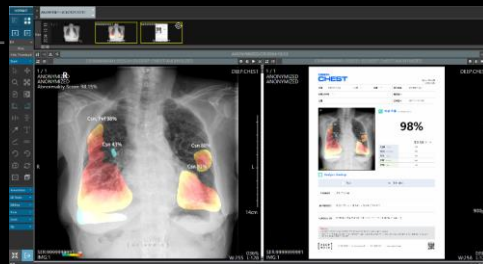
1. The cardiomeastinal silhouette appears normal.
2. No pleural effusion is identified.
3. The heart size is within normal limits.
4. No focal consolidation or consolidation pattern is observed.

DEEP:CHEST-XR-03



- 폐렴, 결핵, 기흉 동시 진단
- 2025년도 NPU 기반 실증으로 판독문 생성 LLM 정확도, 성능 유지를 확인

DEEP:LUNG-LN-02



- AI 기반 폐결절 검출 보조 SW
- 진단 문구, 결절 크기 등 의료인 편의성 목적 세부정보 제공

Douzone – ERP 내 AI 기능 실증 및 코드 어시스턴트 실증

실증업체



ERP



회계



인사



물류

그룹웨어



결재



메일

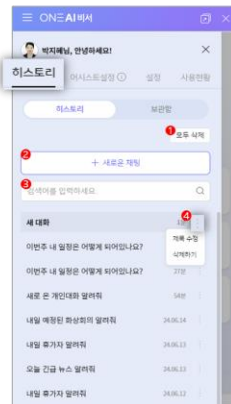


메신저

코드 어시스턴트



powered by **FURIOSA**



WISEnut - NPU 기반 On-prem AI Appliance: Zero Installation AI Platform 실증



WISEnut AI Appliance

NPU 기반 On-prem AI Platform

"설치가 아닌 연결만 하면 되는 AI"

NPU SERVER



GPU 동급 성능

총 소유 비용(TCO) 절감
에너지 사용량과 랙 수량 감소
공랭식 데이터센터와 호환 가능

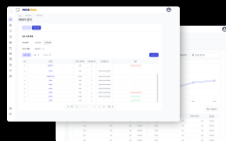
WISEnut의 대표 Agent S/W 탑재

WISEiRAG WISELLOA

바로 쓸 수 있는 챗 인터페이스

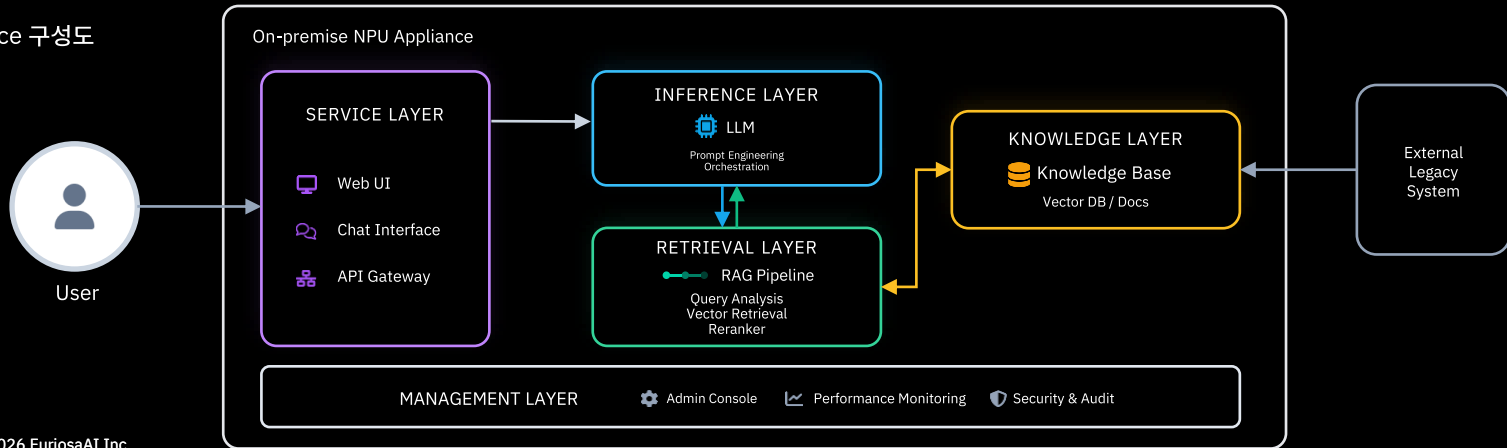


쉽게 사용하는 관리도구

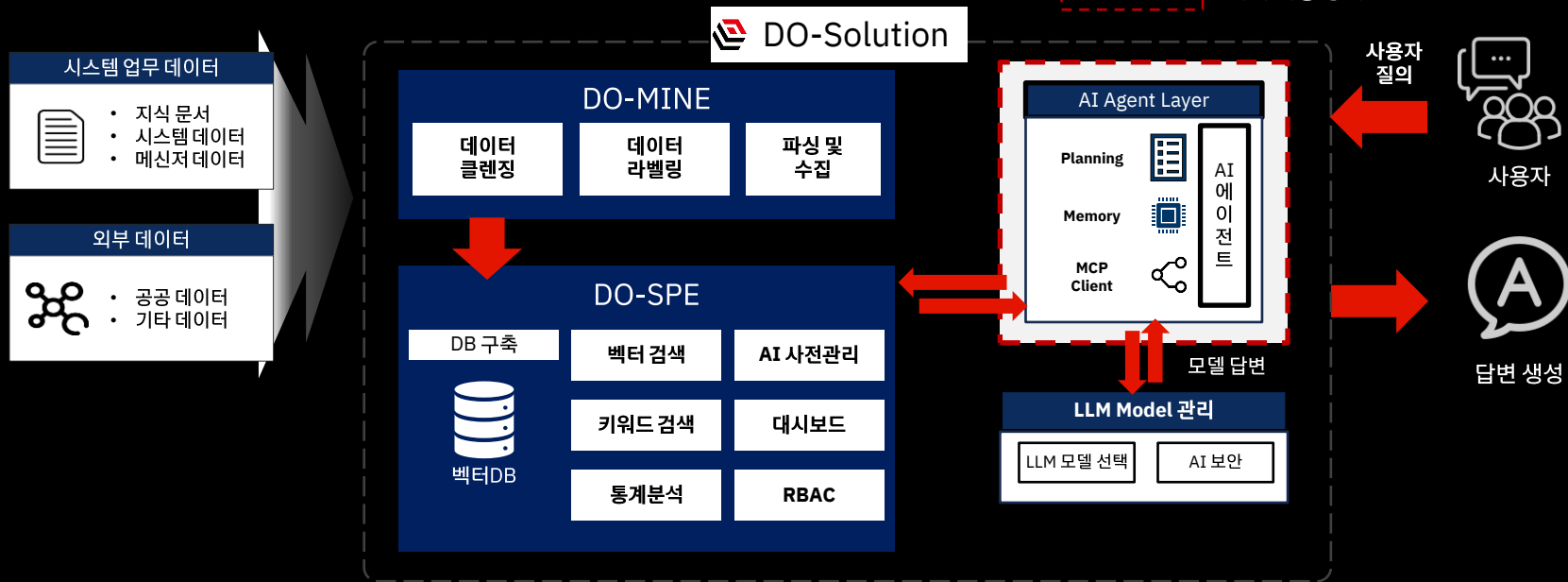


상담·검색·업무자동화까지 다양한 활용처에 간편하고 부담없이 도입

AI Appliance 구성도



로이드케이 – RAG 기반 공공기관 AI Agent 업무지원 시스템 사례



기관 데이터를 AI 활용에 맞게 정제·DB화하고, AI Agent가 참조 가능한 검색 환경을 구성
FuriosaAI RNGD는 Agent Layer의 Planning·Memory 등 핵심 LLM 연산을 하드웨어로 가속

CODEVISION – 제조 공정 NPU 실증 : 비전 불량 검출 + 제조 특화 LLM

최종 수요처

자동차
부품

전기차 아우터캔

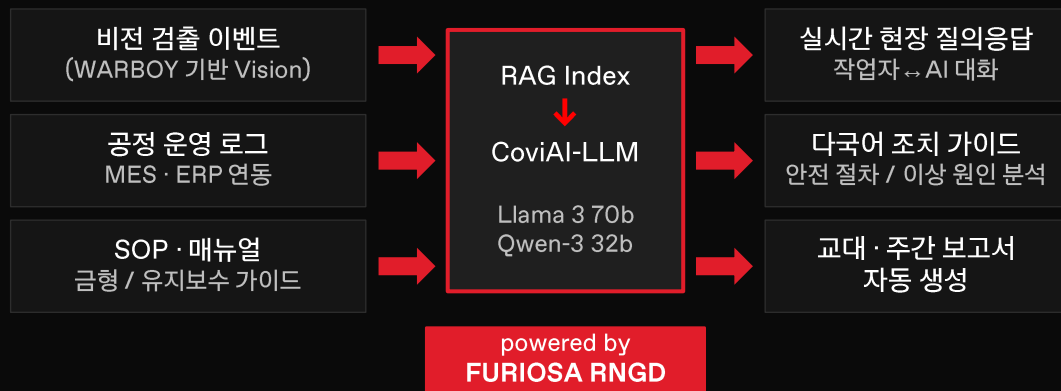
반도체
부품

반도체 컨택트 핀 미세공정

철강가공

코일 강판 (인도네시아)

End-to-End Workflow



Key Integration

- 비전 검출 (이벤트 / 로그 / SOP)
- RNGD LLM의 RAG 인덱스로 직접 입력
- 현장 이슈 발생 시 즉시 대응 가능한 NPU 기반 스마트 제조 솔루션 실증

1차년도 · NPU 기반 비전 AI 불량 검출 (완료)

- RTX 4060 대비 처리속도 192%, 전력 효율 85% 절감
- YOLOv8 INT8 양자화 · AI-EdgeBox + ML-Ops 파이프라인 현장 가동
- KOLAS 공인 제3자 검증 완료

2차년도 · NPU 기반 제조 특화 LLM 솔루션 (진행)

- RNGD 8장 Appliance Server · FP8 양자화
- Llama 3 70b + Qwen-3 32b · PEFT / LoRA 미세튜닝
- vs H100 대비 전성비 RNGD 0.04/0.09 FPS/W (↑)
- 현장 6개월 연속 실증 · TTA / KOLAS 검증 예정



iAcloud – RNGD 기반 운영 및 개발환경 생성부터 조직 Agent 운영 End-to End 플랫폼 구축

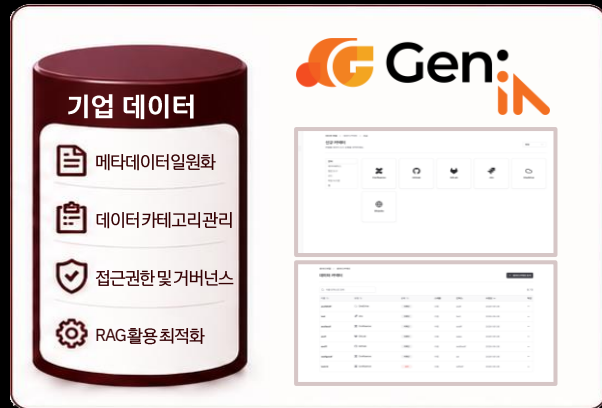
3. 퓨리오사AI RNGD로 배포한 LLM 서버 관리, 접근제어



에이전트 운영



2. LLM 활용 기업 데이터 메타 일원화 및 RAG



1. NPU 연결 가상머신 환경 생성



로앤컴퍼니 - NPU 기반 온프레미스 법률 AI 플랫폼 BriefCase 실증

실증 협력

주관 로앤컴퍼니 | 참여 U社 | 주요 H社 | 하드웨어 FuriosaAI

01 업무 최적화 Workflow

기록 검토 → 리서치 → 서면 작성을 단일 화면에서 수행하는 법률 사건 분석 워크플로우

STEP 1 · 분석

사건 기록 검토

수백 페이지 기록 업로드 → 타임라인·이해관계인 자동 추출



STEP 2 · 리서치

자료 검색 & 질의응답

자연어 검색 + Reference Highlighting으로 근거 원문 확인



STEP 3 · 작성

서면 초안 작성

AI 초안 생성·문장 자동완성·Inline 문단 확장/축소

핵심 기능

시각화

타임라인 자동 추출

시각화

이해관계인 관계도

분석

사건 자동 분석 리포트

검색

화면 전환 없는 판례 검색

작성

버전 관리·자동 저장

협업

팀 관리·활동 피드

02 On-Prem AI Architecture

변호사 · 법무팀

폐쇄 망 분리 환경 내 단일 플랫폼 접속 · 데이터 외부 유출 차단

INFERENCE LAYER

Legal-LLM

법률 특화 LLM

사건 기록 분석·법률 질의응답·서면 초안 작성 특화 추론



RETRIEVAL LAYER

PathRAG 추론 엔진

정확도 95%

지식그래프 연결경로 최적화 · 인용 법률 지식 검증



KNOWLEDGE LAYER

법률 지식 데이터베이스

188만건

판례·결정례·유권해석·행정심판례 등 6종 / 188만건

Furiosa RNGD 워크스테이션

powered by

FURIOSA

국산 NPU · 고효율 온프레미스 구성

망분리

내부망 설치

암호화

SSL/TLS·방화벽 화이트리스트

접근 제어 · 감사

권한 분리 API 호출 로깅

03 실증 성과 & 사업화

95

%

질의응답 정확도
(85%→95%)

90

점

전문가 만족도
(목표 80점)

16

명

동시 사용자 / 2분 이내
(목표 대비 229%)

100

%

서비스 안정성
(폐쇄망 운영)

성능 검증 결과

LLM-as-Judge 총점

358→418점 +17%

질의응답 정확도

85→95% +10%p

문서 작성 보조 수용률

28% 목표 25%

Two-Track 사업화 전략

SuperLawyer

SaaS 구독형 · 개인/중소 로펌
기능 즉시 업데이트

BriefCase

On-Premise 구독형 · 공공/금융
망분리 환경 대응

RNGD 적용 영역

법률 특화 LLM 추론과 RAG 질의응답을 내부망에서 고효율 구동